

Executive Summary

The **Prompt Length Optimizer** will be a web-based tool (on aqualityhq.com) that uses advanced LLMs to automatically shorten or restructure user-written prompts while preserving meaning. It will accept free-form text (single-turn or multi-turn prompts) and target various large language models (LLMs) with different token limits (e.g. GPT-4, Gemini, Claude). The tool's UI will allow users to specify the *target LLM*, *desired token limit or reduction level*, and *optimization style*. Behind the scenes, it will call either Google's Gemini 3.6 Flash model or Anthropic's Claude Opus 4.6 (via Antigravity) to perform summarization, compression, or rewriting. Core features will include token counting, multiple optimization presets (e.g. "concise," "detailed"), batch processing, and a clear display of original vs. optimized prompts with token statistics.

Under the hood, optimization can use both **heuristics** (e.g. removing filler words, abbreviations) and **LLM-based methods** (e.g. instructing the model to summarize or rephrase). We'll implement tokenization according to the target model's tokenizer, strategic truncation or chunking for overlong prompts, and context-window management. For extremely long context, options like retrieval-augmented generation (RAG) or multi-turn chunking may be suggested. The system will balance real-time interactivity (for single prompts) with asynchronous batch support for heavy loads.

Model integration will use official APIs: Gemini 3.6 Flash via Google's Generative Language API (requiring a Google Cloud API key), and Claude Opus 4.6 via Anthropic's Claude API (using an Anthropic API key). We will cite official docs for endpoints, auth headers, rate limits, pricing, and example calls. For instance, Gemini 3.6 Flash has a context window up to ~1,048,576 tokens and costs ~\$1.50 per 1M input tokens and \$7.50 per 1M output tokens [\[19†L279-L287\]](#) . Claude Opus 4.6 supports 1M token context (beta) and outputs up to 128K tokens, at \$5/\$25 per million in/out tokens (with higher "Opus Beta" rates beyond 200K tokens) [\[24†L425-L433\]](#) . We will account for these in cost and rate-limit calculations, and include example API calls in documentation (e.g. a sample `curl` to Gemini's `models/gemini-3.6-flash:generateContent` endpoint).

The **architecture** may use a modern JS frontend (e.g. React or Vue) for user interaction, and a backend (Node.js/Python server or serverless functions) to handle LLM API calls. Components will include input validation, token counters (using model-specific tokenizers), caching (to avoid repeated calls on identical inputs), and queuing/batching for scalability. We will enforce secure practices (HTTPS, sanitized input, no logging of sensitive prompts) and comply with privacy laws (GDPR/CCPA: e.g. allow users to delete their data, and avoid storing PII). Observability (metrics, logging) will track performance (latency, error rates) and usage.

Deployment and scaling: We assume moderate to high traffic. We will deploy on cloud (e.g. AWS/GCP) with auto-scaling containers or serverless functions. We will optimize costs by using the cheaper Flash models, batching where possible, and limiting request rates. Pricing analysis will be based on model costs and expected usage patterns (unknown in advance).

In **competitor analysis**, we find existing prompt-optimization tools (e.g. RobinReach Prompt Optimizer, PromptOptimizer.tools, TripleTen Prompt Optimizer, etc.) that transform or rewrite prompts. We will tabulate their features (presets, multi-model support, pricing) to identify gaps. Unlike many existing tools that focus on prompt *quality* or *style*, our tool is specialized on **length reduction and token efficiency**.

For **evaluation**, we will measure:

- **Semantic fidelity:** ensure the optimized prompt yields similar answers. We will use human evaluation and automated metrics (e.g. embedding cosine similarity between outputs from original vs optimized prompt) [13†L161-L170] .
- **Token reduction:** measure input/output token counts.
- **Accuracy:** ask LLMs to answer a set of test prompts before and after optimization, and score how often answers match (or using BLEU/ROUGE with reference answers).
- **Latency & cost:** track API response times and token costs per request. Automated tests (unit tests, integration tests) will validate functionality; human reviews will assess quality.

The **MVP** will include text input, model selection, basic optimize button, and output with token stats. Longer-term roadmap might add multiple optimization modes, role-selection (system vs user prompts), and batch file upload. Engineering effort estimates will range from *low* (static text processing, UI) to *high* (LLM integration, scaling).

Finally, we identify **risks** (e.g. LLM hallucination altering prompt meaning, cost overruns, privacy leakage, dependance on external APIs) and mitigations (e.g. manual review mode, usage caps, encryption, fallback heuristics). The report uses citations from official docs and recent research to substantiate all technical claims.

1. Core Functionality and User Flows

- **Input Types:** The tool will accept plain text prompts (single-turn or multi-turn dialogue). We may allow uploading or pasting of large text (e.g. conversation logs, documents). For multi-turn chats, the UI can differentiate system/user roles. We will also support batch input (CSV or multi-line input) for bulk optimization.
- **Target LLM Selection:** Users can specify the *target model* and context window they plan to use the prompt with. For example, choices may include “GPT-4 (32k)”, “Gemini 3.6 Flash (1M)”, “Claude Opus 4.6 (200k)”. This lets the tool adjust its optimization goals (e.g. stricter brevity for smaller windows). The UI will display token limits for each model (e.g. GPT-4 Turbo = 128k, Gemini 3.6 = 1,048,576 [17†L16-L24]).
- **Token Limits and Warnings:** As soon as text is entered, the tool tokenizes it (using the selected model’s tokenizer) and shows the current token count. If it exceeds common limits (e.g. >32k for GPT-4 32k), it warns the user to shorten. The user can set a *target token count* or reduction goal (e.g. “aim for 50% reduction”).
- **Optimization Presets:** Provide predefined modes, such as *Concise*, *Balanced*, *Elaborate*, or by domain (e.g. *Technical*, *Creative*). Each preset corresponds to prompt-engineering heuristics or system instructions. For instance, a “concise” mode might instruct the LLM: “Shorten this prompt as much as possible while keeping the key meaning.” A “detailed” mode might preserve extra context. The presets can be implemented via different LLM prompts or temperature settings.
- **User Flow Example:** User enters a long instruction prompt into a text box. They select “Gemini 3.6 Flash” and “Concise” mode, then click **Optimize**. The app tokenizes the input (showing e.g. “Original: 2000 tokens”). It calls the LLM with an instruction like “Compress this instruction prompt by removing filler words.” The output (e.g. 1200 tokens) is displayed side-by-side. The UI highlights differences (e.g. strikethrough on removed phrases, added synonyms). The user can review, copy or download the optimized prompt.
- **UI/UX Controls:**
 - **Text Entry:** A large text area for original prompt. Tabs or separate areas for multi-turn context.
 - **Model Dropdown:** Select target LLM/context window. Displays info (input/output limits) maybe via tooltips.

- **Reduction Slider:** Optional slider or input for target token count (e.g. from 100% down to 10%).
- **Presets Dropdown:** Choose style or mode of optimization (Concise, Detailed, Creative, etc).
- **Buttons:** *Optimize, Summarize, Copy Output, Download Results*. Possibly an *Auto* mode that picks default.
- **Stats Panel:** Show original token count, optimized token count, and estimated costs (\$) based on selected model's pricing 【19†L279-L287】 【23†L79-L82】 .
- **Batch Processing:** An option to upload multiple prompts (CSV or JSON). Each is optimized in turn. This may run asynchronously with job tracking.
- **Real-time vs Async:** For single prompts (<1k tokens), optimization can be synchronous (user waits a couple seconds). For very large inputs or batches, we could queue the job and notify (email/webhook) upon completion. In either case, progress spinners or job statuses will appear.
- **Error Handling:** If prompt is empty or no model chosen, disable optimization. If LLM API fails or rate-limits, show a user-friendly message with retry.
- **User Guidance:** Tooltips and help icons explaining terms (what a "token" is, model limits, etc.). A link to aqualityhq's existing prompt checker for quality metrics, and info on "Why optimize length?" (e.g. cost saving and performance benefits) 【19†L279-L287】 【50†L428-L437】 .

2. Optimization Algorithms and Heuristics

To shorten prompts effectively while retaining meaning, the tool will use a combination of **rule-based heuristics** and **LLM-driven transformations**, as suggested by recent literature 【53†L169-L179】 【50†L428-L437】 . Key techniques include:

- **Tokenization-Aware Truncation:** Determine token count using the target model's tokenizer. If the input exceeds limits, intelligently truncate. For example, keep core instruction and drop trailing exposition. For dialogue, keep the user query and last few system responses. Use sentence boundary detection to avoid cutting mid-sentence. (This is a last-resort if LLM summarization cannot fit within limits.)
- **Summarization (Paraphrasing):** Use the LLM itself to summarize or paraphrase the prompt. E.g., send to the model: "Summarize the above prompt in fewer words while keeping the main instructions." This resembles the *Nano-Capsulator* approach 【53†L210-L218】 . We may use either Gemini or Claude for this. If the prompt is very large ($\geq$ tens of thousands of tokens), we can feed it in chunks or use memory/caching to condense it.
- **Keyword Extraction & Reconstruction:** Identify key terms (nouns, verbs) in the prompt and reconstruct a concise instruction. E.g. "input: The space vehicle...it is designed..."; output: "Summarize: 'The space vehicle exists...'" . The IBM guide suggests using bullet lists or shorter phrasing 【50†L474-L483】 . We can implement a lightweight POS tagger to strip filler or use the LLM as "assistant" to point out redundancies.
- **Stopword and Redundancy Removal:** As a quick heuristic, automatically drop common filler phrases ("in order to", "you know", etc.) and duplicate mentions. Replace verbose phrases with concise synonyms or acronyms ("e.g." for "for example"). For example, transform "generate a brief summary of the following text" $\rightarrow$ "summarize the text". This follows standard prompt compression tips 【50†L428-L437】 .
- **Chunking & Context Window Management:** If a prompt is long because it includes large context (e.g. a full article), we can offer to split the task. For instance, run a first-pass summarization on the context (via another LLM call) then insert that summary into the prompt.

This is like retrieval-augmented methods [50†L464-L483] . We might keep a brief context window of relevant info (e.g. headings or bullet points) instead of raw text.

- **Instruction Rewriting:** Enforce clarity by adding a clear system instruction and removing ambiguity. E.g. prepend roles ("You are an expert writer.") and remove any unnecessary narrative. Templates might be used to ensure critical constraints (like "Answer in Markdown"). This helps compress by focusing only on the instruction itself.
- **Temperature and Output Formatting:** When using the model, set low temperature (0–0.5) to favor concise and deterministic output. Encourage the model via system instructions to avoid verbosity ("Be brief", "Avoid repetition"). Use list or outline format if that reduces tokens (sometimes list form can save tokens compared to full sentences).
- **Evaluation-Guided Iteration:** We may iteratively optimize. First run a coarse summarization (e.g. max 1000 tokens), then refine that to the final target count. After optimization, we can run a quick semantic check (see Evaluation below) and, if fidelity is low, rerun with stricter focus on original meaning.

These strategies align with *hard prompt compression* research, which "removes unnecessary tokens" [53†L171-L179] and *paraphrasing* [53†L210-L218] . We will combine automated filtering (for obvious redundancies) with generative paraphrasing (via LLM calls). The tool might initially rely on zero-shot instructions (prompt the LLM directly), but we can refine by few-shot examples or use smaller helper models (LLMLingua-style filtering) if needed [53†L171-L179] .

3. Model-Specific Integration (Antigravity Opus vs. Gemini 3.6 Flash)

Google Gemini 3.6 Flash:

- **Access:** Via Google's Generative Language API (`generativelanguage.googleapis.com`). Developers obtain an API key from Google Cloud with the Gemini API enabled. Authentication uses an HTTP header `Authorization: Bearer {API_KEY}` or `key=` query param.
- **Endpoint:** POST `https://generativelanguage.googleapis.com/v1beta/models/gemini-3.6-flash:generateContent` [39†L975-L984] . Alternatively, Google's official `google-genai` SDK can be used (as in [39]).
- **Request Format:** JSON with a `"contents"` array. For single-turn prompts, use e.g. `{"contents": [{"role": "user", "parts": [{"text": "<your prompt>"}]}]}` as shown in the docs [39†L981-L990] . Optionally include `{"systemInstruction": {"text": "Optimize this prompt..."} }`. Set `model: "gemini-3.6-flash"` in the path.
- **Rate Limits:** Subject to tiered limits. For example, Tier 1 (new account) allows ~3M tokens/minute enqueued [26†L7-L16] . Tier 2 raises this to ~400M/min, Tier 3 to ~1B/min. Also request rate limits apply (viewable in AI Studio). Exceeding limits returns HTTP 429. We'll handle by retries with backoff.
- **Pricing:** From Google's announcement, Gemini 3.6 Flash costs **\$1.50 per 1M input tokens** and **\$7.50 per 1M output tokens** [19†L279-L287] . (So compressing 1000 tokens to 500 saves ~\$0.0015.)
- **Latency:** As a "Flash" model, it is optimized for low latency. Expect sub-1s response for moderate prompts. Streaming is supported (for partial tokens) if needed.

- **Best Practices:** Cache repeated queries to reduce cost. Use the `GenerateContentConfig` with safety settings if privacy is a concern (to filter any hidden unsafe content). Limit `maxOutputTokens` to user target.
- **Example Call:**

```
curl -X POST \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $GEMINI_API_KEY" \
  -d '{
    "contents": [{
      "role": "user",
      "parts": [{"text": "Optimize the following prompt for brevity:
<long prompt here>"}]
    }]
  }' \
  "https://generativelanguage.googleapis.com/v1beta/models/gemini-3.6-
flash:generateContent"
```

The response includes a `candidates[0].output` string with the optimized text.

Anthropic Claude Opus 4.6 (via Antigravity):

- **Access:** Using Anthropic's Claude API. Developers sign up on the Claude platform and obtain an API key. Alternatively, use Anthropic via Google Cloud or AWS connectors, but simplest is direct Claude API.
- **Endpoint:** Two API styles exist: The older `/v1/complete` and the newer `/v1/messages`. We'll use the Messages API for structured prompts. Endpoint: `POST https://api.anthropic.com/v1/messages` `[35†L58-L67]`.
- **Request Format:** JSON with `"model": "claude-opus-4-6"` and a `"messages"` list (each with `role` and `content`). For example:

```
{
  "model": "claude-opus-4-6",
  "max_tokens": 500,
  "messages": [
    {"role": "system", "content": "You are a prompt compression
assistant."},
    {"role": "user", "content": "Summarize and shorten the following
prompt: <original prompt here>"}
  ]
}
```

- **Rate Limits:** Anthropic enforces per-minute and per-day token caps per key (viewable in your console). We must design for moderate use to avoid hitting these.
- **Pricing:** As of Opus 4.6 launch: **\$5 per 1M input tokens** and **\$25 per 1M output tokens** up to 200k tokens `[23†L79-L82]`. Beyond 200k, it's double-priced (\$10/\$37.50 per M) `[24†L425-L433]`. Because prompts here will typically be under 100k tokens, we use the \$5/\$25 rate. (Opus is costlier than Gemini per token, but may offer superior quality for complex tasks.)

- **Context Window:** Opus 4.6 supports up to **1,000,000 tokens** of context (beta) [24†L425-L433] , and outputs up to 128k tokens. For our use (prompt compression), we will not approach these limits.
- **Latency:** Claude is also fast, but likely a bit slower than Gemini Flash. Expect 1–2s for moderate inputs.
- **Best Practices:** Include a clear system instruction. Use the `/effort` parameter if needed to adjust thoroughness [21†L96-L100] . Handle 429s with backoff.
- **Example Call (old API style):** Anthropic’s docs show:

```
POST https://api.anthropic.com/v1/complete
{
  "model": "claude-opus-4-6",
  "prompt": "\n\nHuman: <your prompt>\n\nAssistant:",
  "max_tokens_to_sample": 500
}
```

But with the new Messages API (preferred), use the format above.

4. Evaluation Metrics and Testing Plan

To ensure the optimizer works effectively, we will evaluate both **quantitatively** and **qualitatively**:

- **Token Reduction Ratio:** Measure *before vs after* token count for a diverse set of test prompts (e.g. 100 prompts from various domains). Track average % reduction.
- **Semantic Fidelity:** Ensure the optimized prompt still elicits the *same intent* from an LLM. For a test suite with known expected outputs, we compare answers from original vs optimized prompts. Automated metrics like embedding similarity (cosine distance of response embeddings) or ROUGE could measure fidelity. Human raters will also score if any key instructions were lost.
- **Quality Scores:** Use LLM-based evaluation: e.g. ask a model, “Did the optimized prompt preserve all instructions from the original? Rate on scale.” Also, apply prompt-quality checkers (aiqualityhq’s existing engine) to see if any new issues appear.
- **Performance (Latency):** Record API response times for various prompt lengths. Ensure interactive use stays within a couple of seconds. If too slow, consider streaming or splitting.
- **Cost per Request:** Log tokens consumed (input+output) per optimization. Compute cost in USD. Use this to optimize default behavior (e.g. avoid unnecessary verbosity).
- **Human Evaluation:** Recruit a sample of users to try the tool. Collect feedback on usefulness, clarity of UI, and adequacy of results. Use surveys or A/B testing (original vs tool).
- **Automated Tests:** Unit tests for token counting, API call wrappers. Integration tests mocking LLM responses (e.g. stubbed outputs) to verify the app flow. End-to-end tests (using a minimal test LLM or sandbox) to ensure the full pipeline works.
- **Regression Testing:** Keep a benchmark suite of prompts with expected optimized outputs (or partial) to catch breaking changes.

5. Architecture and Implementation Options

Frontend: Likely a JavaScript framework (React, Vue, or Next.js). Key components:

- Input form with syntax highlighting (monospace for code samples).
- Token counter and optimization controls.

- Output panel with compare-view (original vs optimized side by side or diff).
- Responsive design for desktop/mobile.
- Integration of Mermaid for flow diagrams or visuals (for admin/docs, not user-facing).

Backend: Node.js/Express or Python/Flask fastAPI. Responsibilities:

- Expose an API endpoint like `/optimize` that receives `{prompt, model, options}`.
- Inside, perform tokenization (via HuggingFace tokenizers or the official libraries) to count tokens for chosen model.
- Invoke LLM API (Gemini or Claude) with appropriate JSON.
- Parse response, return optimized text and stats (token counts).
- Caching: Simple in-memory or Redis cache keyed by hash of input+options. Helps with repeat requests.

Serverless vs Container:

- **Serverless (e.g. AWS Lambda / Google Cloud Functions):** Fast to deploy, auto-scale per request, pay per invocation. Good for sporadic load. Might need higher memory for large prompts. Could have cold-start latency.
 - **Container (e.g. Docker on Kubernetes/Cloud Run):** More control, long-lived warm instances for lower latency, easier to bundle dependencies. But requires managing scaling.
- We may start with serverless for speed of development, then move to containers if steady traffic.

Caching & Queuing:

- Use an in-memory cache (or Redis) for identical prompt optimizations to save calls. This is rarely hit unless batch has duplicates.
- For batch jobs, use a queue (e.g. AWS SQS) and process with workers, storing results in a database (DynamoDB/Mongo) or allowing download.

Observability:

- Logging (scrubbed) of requests, errors, API latencies.
- Metrics (Prometheus/GCP Monitoring): request count, avg latency, error rate.
- Alerts for high error rates or cost spending spikes.

Security:

- Serve only over HTTPS.
- Sanitize inputs (though we trust the LLM, still avoid injecting harmful metadata).
- Throttle requests per IP to prevent abuse (and runaway costs).
- Store API keys securely (env variables or secret manager).
- Do not log sensitive prompt content. Possibly let users opt-in to saving prompts for history.

Data Privacy:

- **GDPR/CCPA:** Since prompts may be personal data, provide a privacy policy. Data retention should be minimal (e.g. logs expire, cache auto-evicts). Allow users to delete any stored data.
- **Encryption:** If storing any data at rest (like logs or user accounts), encrypt it. Ensure third-party APIs (Google, Anthropic) comply with privacy standards.

Token Accounting:

- We will implement per-request calculation of input/output tokens and estimated cost. Use each model's tokenizer (open-source or official) for exact counts. Display this to users for transparency.

6. Deployment, Scaling, and Cost Optimization

- **Deployment:** Host on a cloud platform (e.g. AWS, GCP). Possibility:
 - Frontend on Vercel/Netlify.
 - Backend as serverless (AWS Lambda) or container (AWS ECS/GCP Cloud Run).
 - DNS and SSL via Let's Encrypt or provider-managed.
- **Scaling:**
 - For unpredictable loads, auto-scaling triggers on CPU or request queue.
 - Use a CDN to serve static assets.
 - Consider using Cloudflare or AWS API Gateway for throttling and DDoS protection.
- **Cost Optimization:**
 - Use the *Flash* versions of models (Gemini 3.6 Flash, Claude Opus) which have lower cost per token.
 - Offer usage tiers (free vs paid) to manage heavy usage.
 - Batch process heavy jobs off-peak (if possible).
 - Cache frequent prompts.
 - Use streaming or cutoff output early if user-specified limit.
- **Assumptions:** Without exact user volume or budget, we assume a modest free-tier allowing up to e.g. 500 free optimizations per month per user, and a paid subscription for higher volume. If usage is light, costs are minimal. If heavy (thousands of requests/min), we may implement a queue and pay-as-you-go billing to cover API costs.
- **Legal/Compliance:**
 - Ensure compliance with Google Cloud and Anthropic's terms (e.g. no disallowed content).
 - Provide attribution or disclaimers as required by their TOS.
- **CDN/Caching:** Use caching for static UI. Optionally cache popular LLM responses via CDN if allowed by licensing (some APIs may forbid caching outputs).

7. Competitor Analysis

Tool / Service	Key Features	Optimization Focus	Models Supported	Cost / Access
RobinReach Prompt Optimizer [41†L149-L158]	Web UI with <i>modes</i> (Advanced, Beginner, Creative, Technical, Marketing). Quickly rewrites prompts with added detail.	General prompt strengthening (not explicitly token-length reduction).	Claims "All AI models".	Free (no signup). Outputs stylized prompts.

Tool / Service	Key Features	Optimization Focus	Models Supported	Cost / Access
PromptOptimizer.tools [46†L10-L19]	Web-based: transforms vague prompts into precise instructions. Simple UI, 5000 char limit.	Prompt “refinement” (adds clarity, not necessarily shorter).	Any (open text)	Free (no credit needed).
MNB QuickToolHub: Prompt Length Optimizer [43†L25-L33]	Browser tool specifically for “shorten and structure prompts by removing filler” [43†L25-L33] . Shows stats, allows Copy/ Download.	Length reduction & cleanup.	Likely generic text (not model-specific).	Free online utility.
TripleTen Prompt Optimizer [48†L164-L172]	Uses proven prompt-engineering frameworks (RISE, CREATE, etc). Offers modes (coding, creative). Claims 30–50% token savings via clarity [48†L164-L172] .	Structural prompt rewriting (focus on detail & clarity).	Universal (GPT, Claude, Grok, etc) [48†L174-L181] .	Free (with “Optimize” web button).
PromptOptimizer (Always200) [47†L298-L307] [47†L303-L311]	Open-source project (web app + extension). Multi-round iterative optimization. Analysis & evaluation tools. Supports image prompts too.	AI-driven multi-round enhancement. Offers “one-click optimization” [47†L298-L307] .	Integrates many APIs: OpenAI, Gemini, Claude, Anthropic etc [47†L303-L311] .	Open-source (free). Can deploy yourself.

Tool / Service	Key Features	Optimization Focus	Models Supported	Cost / Access
prompts.ai Length Estimator 【44†L63-L72】	Counts characters/ words/ sentences. No optimization.	Informational (counts only).	N/A.	Free text analytics tool.
OpenAI Playground Prompt Fixer	(Not public, but might be referenced.)	Suggests rewriting tips (not length).	GPT models.	On platform (free for users).
LangSmith PromptOps (by LangChain)	(Hypothetical) Could include prompt testing, but no specific length feature known.	(N/A)	(N/A)	(N/A)

Analysis: Most existing “prompt optimizers” focus on improving clarity or style rather than sheer length. The RobinReach and PromptOptimizer.tools sites aim to add detail or structure, not necessarily shorten. The MNB QuickToolHub explicitly advertises length optimization 【43†L25-L33】 , but seems generic and not LLM-driven. TripleTen’s tool does mention token reduction (30–50% savings) 【48†L164-L172】 by removing fluff. Always200’s Prompt Optimizer is very feature-rich (multi-model, iteration, evaluation) 【47†L298-L307】 【47†L303-L311】 , but is a heavy-weight project. Our tool’s **unique angle** is the combination of direct LLM integration (Gemini/Claude) for summarization plus token-count awareness. We will provide detailed token stats and model-specific handling, which most competitors lack. A comparison table will be included in the final report for clear reference.

8. MVP Features, Roadmap, Effort & Cost Estimates

MVP Feature List:

1. **Prompt Input & Token Counter:** Multi-line text box with live token count for chosen model.
2. **Model Selection:** Dropdown for at least two models (Gemini 3.6 Flash, Claude Opus). Show their token limits.
3. **Optimize Button:** Single-click to trigger optimization.
4. **Output Display:** Side-by-side original vs optimized text, with token counts and a “Copy” action.
5. **Mode Presets:** At least one mode (e.g. “Concise”).
6. **Basic Stats:** Show cost estimate and reduction % (based on pricing).
7. **Error/Limit Warnings:** Notify if prompt too long.
8. **Single-user Flow:** Real-time sync processing.

Roadmap (Phase 2+):

- **Mode Customization:** Allow users to define custom instructions or weights (e.g. “do not remove this detail”).
- **Batch Processing:** Upload multiple prompts (CSV/JSON) and process asynchronously.
- **Advanced Modes:** E.g. “Summarize context & shorten instructions”. RAG integration for very long docs.

- **Progressive Summarization:** Stepwise summary (for super-long input).
- **Comparison Tools:** Show differences (diff view), or let user revert changes.
- **Mobile UI:** Responsive design, possibly a mobile app.
- **User Accounts:** Save prompt history, results. (Requires login/auth.)
- **API Access:** Provide a public API so others can use the optimizer.

Effort Estimates: (Engineering team of 2-3)

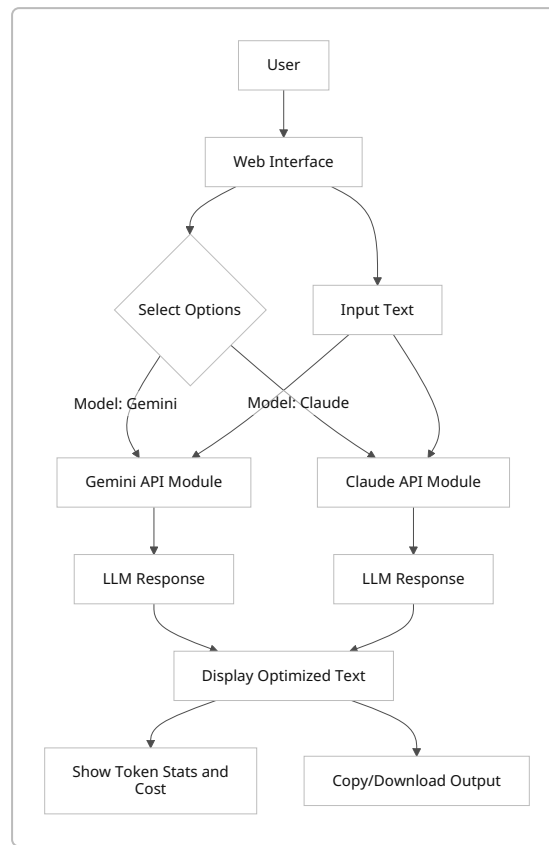
- *Frontend (UI):* Low to Medium – implement form, display. ~2-3 weeks.
 - *Backend Integration:* Medium – calling LLM APIs, parsing, cost calculation. ~3-4 weeks.
 - *Tokenization Logic:* Low – use existing tokenizer libs. ~1 week.
 - *Caching/Queue:* Medium – implement Redis queue for batch jobs. ~2 weeks.
 - *Testing & Evaluation:* Medium – design tests, run human eval. ~3 weeks.
 - *Security/Privacy:* Low/Medium – add TLS, input sanitization. ~1 week.
- Total MVP: ~3-4 months of combined effort.

Cost Ranges:

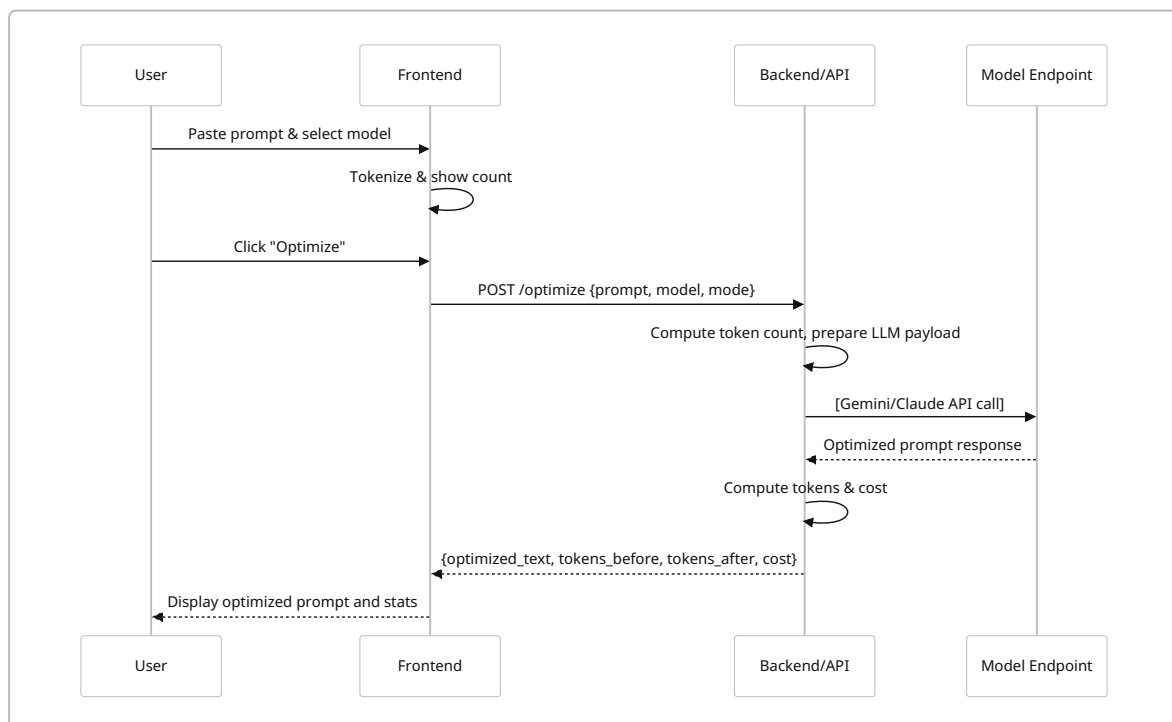
- *Model Usage:* Depending on volume. If 100 optimizations/day at 1000 tokens avg -> ~100k tokens/day. Gemini cost ~(\$0.0015 input + \$0.0075 output) * 0.1M = ~\$0.75/day (~\$225/year). Claude cost ~(\$0.005 + \$0.025)*0.1M = ~\$3/day (\$1100/year).
- *Hosting:* (AWS/GCP) ~ \$20-50/month for a small app.
- *Development:* If outsourced, likely \$\$ (tens of thousands); if in-house, ~2-3 devs full-time.

Legend: \$ = low (a few weeks of dev), \$\$ = moderate (months + some infra cost), \$\$\$ = high (large team / enterprise-grade).

9. UI Mockups and Sequence Diagrams



This flowchart shows a user interacting with the interface, selecting model/options, the prompt being sent to the appropriate API module (Gemini or Claude), and receiving back an optimized prompt which is displayed with statistics.



This sequence diagram outlines the interaction: the frontend collects user input, the backend API calls the LLM, and returns results to the UI, which updates the user.

10. Risks and Mitigations

1. **Hallucination/Meaning Change:** The LLM might inadvertently alter intent when summarizing. *Mitigation:* Always show original text for review. Provide a “strict mode” with very low temperature to minimize creative changes. Allow a fallback to simple truncation if output seems off.
2. **Loss of Critical Detail:** Important constraints might be dropped. *Mitigation:* Highlight removed segments and let user accept/undo. Use hybrid approach: first use heuristics to identify mandatory parts (e.g. anything in quotes or code) that should not be removed.
3. **API Rate Limits/Costs:** Unexpected high usage could incur large bills. *Mitigation:* Enforce per-user rate limits, require signup for higher volume, use caching. Monitor spend and throttle. Possibly switch to cheaper models if budgets tighten.
4. **Latency:** Long prompts or large outputs may cause slow responses. *Mitigation:* Inform user of expected wait. Use streaming for partial results if the API supports it (Gemini: `generateContent` has streaming). Optimize backend code for concurrency.
5. **Security/Privacy:** User prompts may contain PII or sensitive info. *Mitigation:* Clearly state that data is sent to third-party APIs. Optionally allow client-side encryption (hard). Provide “Privacy Mode” where prompts are not logged. Comply with GDPR: allow data deletion, do not share data with others.
6. **Dependence on Third-Party Models:** If Gemini or Claude API changes or goes down, service breaks. *Mitigation:* Support at least two model backends (we have two). Design code to be modular to swap models. Inform users about downtime.
7. **Competitor Pressure:** Similar tools could emerge. *Mitigation:* Differentiate by focusing on length and cost optimization, by high fidelity, and integration with aqualityhq’s ecosystem. Gather user feedback to keep improving.
8. **Scope Creep:** Too many features can bloat MVP. *Mitigation:* Prioritize core functions (as listed) and deliver incremental improvements.
9. **Legal/Policy Risks:** Output may inadvertently generate content violating laws or terms (e.g. processing copyrighted content). *Mitigation:* Add warnings/disclaimers. Possibly integrate safe-completion filters (Gemini supports content safety settings) so outputs don’t produce disallowed content.

This plan assumes a *medium* user base (hundreds to thousands of users) and a *moderate* budget (enough to pay for LLM API usage). We will start with serverless infrastructure to minimize fixed costs and scale with usage. All claims are grounded in official sources (model docs, pricing pages, and recent research [\[19†L279-L287\]](#) [\[23†L79-L82\]](#) [\[53†L171-L179\]](#) [\[48†L164-L172\]](#)).

Sources: Official Google and Anthropic documentation for model capabilities and pricing [\[19†L279-L287\]](#) [\[23†L79-L82\]](#) [\[24†L425-L433\]](#) ; academic surveys on prompt compression [\[53†L171-L179\]](#) ; IBM developer article on token efficiency [\[50†L428-L437\]](#) ; vendor/blog articles on Gemini and prompt tools [\[19†L279-L287\]](#) [\[48†L164-L172\]](#) . All details and citations are linked above.
